

Probeklausur V3 — 06.03.2026

Aufgabe 1 (47.25 Punkte)

Implementiere die Klassen

- **Boardable** (1.5 Punkte),
- **BoardingGroup** (3.5 Punkte),
- **Passenger** (3.5 Punkte),
- **Seat** (10.0 Punkte),
- **CrewSeat** (5.75 Punkte),
- **ExitSeat** (6.0 Punkte),
- **Airplane** (12.0 Punkte) und
- **ExamTask** (5.0 Punkte)

entsprechend dem Klassendiagramm.

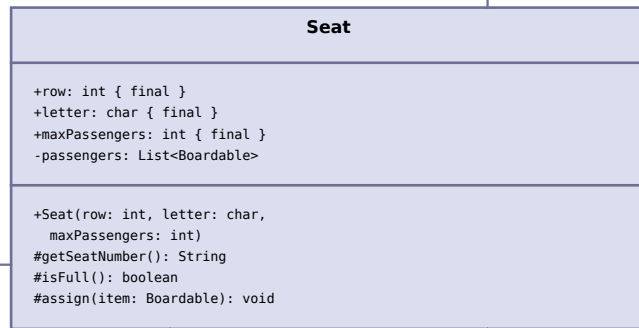
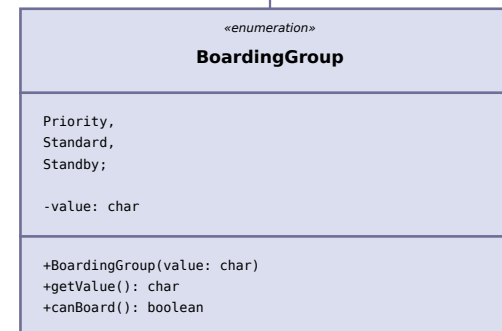
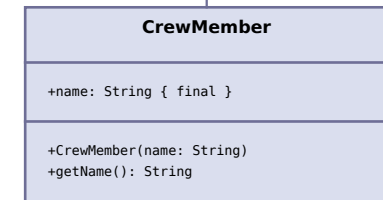
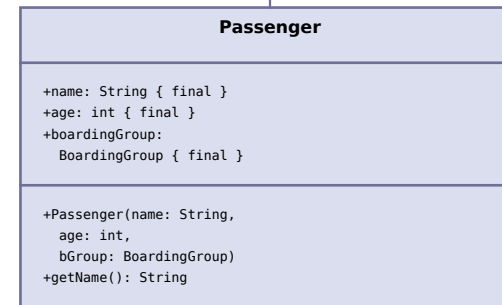
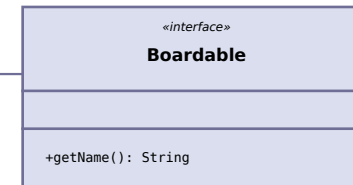
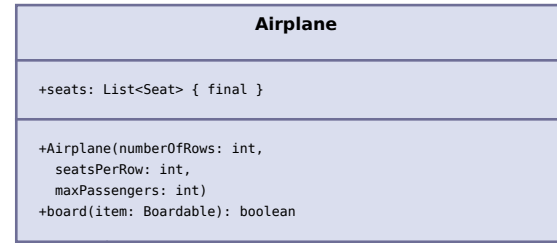
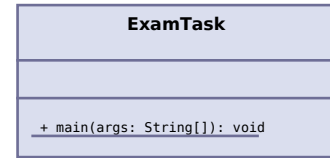
Befolge alle Hinweise bei der Implementierung!

Die Klasse CrewMember ist gegeben und muss nicht implementiert werden.

Glossar

Englisch	Deutsch
airplane	Flugzeug
assign	zuweisen
board	einsteigen
boardable	einsteigefaeig
boarding group	Boarding-Gruppe
crew member	Besatzungsmitglied
crew seat	Crew-Sitz
exit seat	Notausgang-Sitz
passenger	Passagier
priority	Prioritaet
seat	Sitz
standby	Nachruecker

Klassendiagramm



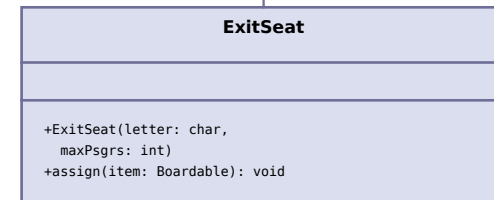
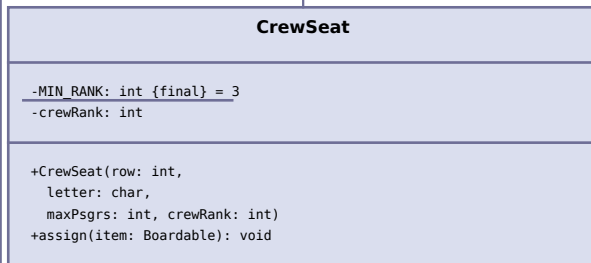
extends

extends

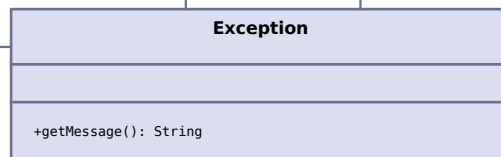
extends

implements

implements



throws



throws

Hinweise zur Klasse Boardable (Einsteigefaeig)

- Die Methode **getName** soll den Namen zurueckgeben.

Hinweise zur Klasse BoardingGroup (Boarding-Gruppe)

- Die Methode **getValue** soll den Wert (value) der Boarding-Gruppe zurueckgeben.
- Die Methode **canBoard** soll true zurueckgeben, wenn die Boarding-Gruppe zum Einsteigen berechtigt ist.

Eine Boarding-Gruppe ist zum Einsteigen berechtigt, sofern es kein Nachruecker (Standby) ist.

Hinweise zur Klasse Passenger (Passagier)

- **Konstruktor** soll alle Attribute initialisieren.
- Die Methode **getName** soll den Namen zurueckgeben.

Hinweise zur Klasse Seat (Sitz)

- **Konstruktor** soll alle Attribute initialisieren.
- Die Methode **getSeatNumber** soll die Sitzkennung als Text zurueckgeben.

Die Sitzkennung besteht aus der Reihenummer und dem Buchstaben (z.B. "4A" fuer Reihe 4, Buchstabe A).

- Die Methode **isFull** soll ermitteln, ob der Sitz voll belegt ist.

Ein Sitz ist voll, sobald die Anzahl der Passagiere die maximale Anzahl (maxPassengers) erreicht hat.

- Die Methode **assign** soll versuchen einen Passagier dem Sitz zuzuweisen.

Ist der Sitz nicht voll, soll der Name des Passagiers und die Sitzkennung auf der Konsole ausgegeben und der Passagier hinzugefuegt werden. Ist der Sitz voll, soll ein Fehler mit der Sitzkennung ausgeloeset werden.

Bsp: "Pilot Mueller auf Sitz 5A"

Hinweise zur Klasse CrewSeat (Crew-Sitz)

- **Konstruktor** soll alle Attribute initialisieren.

Der Crew-Sitz hat extra Ausruestung und somit 1 Platz weniger als die maximale Anzahl.

- Die Methode **assign** soll versuchen, einen Passagier im Crew-Sitz zu platzieren.

Nur Besatzungsmitglieder (CrewMember) duerfen im Crew-Sitz platziert werden. Ist der Crew-Rang (crewRank) kleiner als der Mindest-Rang (MIN_RANK), darf niemand platziert werden.

Kann der Passagier nicht platziert werden, soll ein Fehler ausgeloeet werden, dass nur Crew in diesem Sitz erlaubt ist.

Hinweise zur Klasse ExitSeat (Notausgang-Sitz)

- **Konstruktor** soll alle Attribute initialisieren.

Der Notausgang-Sitz soll immer in der untersten Reihe sein (Reihe 1).

- Die Methode **assign** soll versuchen, einen Passagier im Notausgang-Sitz zu platzieren.

Nur Passagiere (Passenger) deren Boarding-Gruppe zum Einsteigen berechtigt ist, duerfen im Notausgang-Sitz platziert werden.

Kann der Passagier nicht platziert werden, soll ein Fehler ausgeloeet werden, dass nur Passagiere mit gueltigem Boarding erlaubt sind.

Hinweise zur Klasse Airplane (Flugzeug)

- **Konstruktor** soll alle Attribute initialisieren.

Jedes Flugzeug hat mehrere Sitze. Es gibt jedoch immer genau einen Crew-Sitz und einen Notausgang-Sitz. Der Konstruktor soll nach der Initialisierung aller Attribute zuerst den Crew-Sitz, dann den Notausgang-Sitz und anschliessend die normalen Sitze erzeugen und der internen Liste hinzufügen.

Die normalen Sitze werden in einem verschachtelten Loop erzeugt (Reihen absteigend, Buchstaben aufsteigend). Der Notausgang-Sitz hat immer die geringste Reihe (Reihe 1, Buchstabe A) und der Crew-Sitz immer die höchste Reihe (Buchstabe A). Der Crew-Sitz hat 4 als Crew-Rang (crewRank).

Beispiel Liste mit numberOfRows = 3, seatsPerRow = 2, maxPassengers = 2:

```
- CrewSeat - Sitz 5A, max. 1 Passagier (2-1)
- ExitSeat - Sitz 1A, max. 2 Passagiere
- Seat     - Sitz 4A, max. 2 Passagiere
- Seat     - Sitz 4B, max. 2 Passagiere
- ...      (Reihen 3, 2 analog)
```

- Die Methode **board** soll einen Passagier platzieren und zurueckgeben, ob es funktioniert hat.

Versuche einen Platz fuer den Passagier im Flugzeug zu finden, indem ueber alle Sitze iteriert wird. Wenn ein Platz gefunden wurde, soll true zurueckgegeben werden.

Wenn der Passagier in einem Sitz nicht platziert werden konnte, soll die Fehlermeldung auf der Konsole ausgegeben und der naechste Sitz versucht werden.

Konnte der Passagier in keinem Sitz platziert werden, soll false zurueckgegeben werden.

Hinweise zur Klasse ExamTask

Es soll ein Flugzeug mit 3 Reihen, 2 Sitzen pro Reihe und maximal 2 Passagieren pro Sitz erstellt werden.

Erstelle ein Besatzungsmitglied mit einem Namen deiner Wahl und versuche es im Flugzeug zu platzieren. Falls kein Platz gefunden wurde, soll dies auf der Konsole ausgegeben werden.

Iteriere ueber alle Boarding-Gruppen. Erstelle fuer jede Boarding-Gruppe einen Passagier mit einem Alter von 30 und versuche ihn im Flugzeug zu platzieren. Verwende den Wert (getValue) der Boarding-Gruppe im Namen des Passagiers.

Falls kein Platz gefunden wurde, soll dies zusammen mit dem Wert der Boarding-Gruppe auf der Konsole ausgegeben werden.

Java API

Klasse	Methode	Statisch	Rueckgabotyp
BoardingGroup	values()	X	BoardingGroup[]
BoardingGroup	name()		String
Exception	getMessage()		String
Boolean	valueOf(s: String)	X	Boolean
Boolean	valueOf(b: boolean)	X	Boolean
Double	valueOf(s: String)	X	Double
Double	valueOf(d: double)	X	Double
Integer	valueOf(s: String)	X	Integer
Integer	valueOf(i: int)	X	Integer
String	charAt(index: int)		char
String	length()		int

Java Collections Framework

Klasse	Methode	Statisch	Rueckgabotyp
ArrayList<T>	add(element: T)		boolean
ArrayList<T>	add(index: int, element: T)		void
ArrayList<T>	contains(element: T)		boolean
ArrayList<T>	get(index: int)		T
ArrayList<T>	remove(index: int)		T
ArrayList<T>	remove(element: T)		boolean
ArrayList<T>	size()		int
Collections	sort(list: List<T>)	X	void
Collections	sort(list, c: Comparator<T>)	X	void

Schnittstellen

Klasse	Methode	Statisch	Rueckgabotyp
Comparable<T>	compareTo(o: T)		int
Comparator<T>	compare(o1: T, o2: T)		int